CMJM Lab — Entry Programming Assessment

General Guidance for Prospective Students
National Taiwan University · CMJM Lab (Prof. Mei-Ju May Chen)

1. Purpose

Before joining the lab, prospective students complete a short programming assessment. Its purpose is **not** to test competitive-programming skill, memorised algorithms, or knowledge of any particular biological topic. You will not be asked to invert a binary tree.

What we are looking for is the most important day-to-day skill in a computational biology lab: **can you take imperfect, real-world data and turn it into something you can defend — and can you tell someone else exactly what you did and why?**

Almost all work in this lab starts from data that someone else generated, in formats that do not quite line up, with problems nobody warned you about. A student who handles that carefully will succeed here regardless of their starting level. A student who produces a confident-looking result from data they never checked will not.

2. Please note: the task is individual

There is no single fixed exam.

The assessment is tailored to each applicant — to your background, your stated interests, and the projects currently running in the lab. Two students joining in the same semester will normally receive different tasks.

This document therefore describes **the kind of thing you will be asked to do and how your work will be judged**, not the task itself. Your own task description is the authoritative document: it will specify the exact scope, the deliverables, and the deadlines. Where the two differ, follow your task description.

3. The kind of work involved

Assessments in this lab are drawn from the same pool of practical skills. Expect some combination of the following, weighted differently depending on your task:

- **Working with real data files.** Loading data you did not create, understanding its structure before manipulating it, and combining information that arrives in several separate files that have to be linked through shared identifiers.
- **Checking your own work.** Real datasets contain errors, inconsistencies, and gaps. You will be expected to look for them rather than assume they are absent, to show the evidence for what you find, to make a reasoned decision about how to handle it, and to say why. There is rarely a single correct handling strategy; there is always a difference between a considered choice and an unexamined one.

- **Explaining yourself in writing.** A short report covering what you did, what you found, what you decided, and what you would do differently with more time. This carries real weight — it is not a formality attached to the code.
- **Possibly some lightweight analysis or modelling.** Some tasks include a simple modelling or statistical component, often with an interest in whether a simpler or cheaper approach performs comparably to a more complex one. Where this appears it is usually optional or secondary, and the emphasis is on sound implementation and honest interpretation rather than on maximising a performance number.

4. Format

- **Take-home**, on your own machine. Not supervised, not timed minute-by-minute.
- You will receive the materials and specification in a shared folder, and submit your work back to the same place.
- The working window is short — normally a small number of days at most. **Your task description states the actual deadlines.**
- If something in your task is genuinely ambiguous, **ask.** A well-posed question is a positive signal, not an admission of weakness. Do not spend hours guessing.

5. Tools

- **Python is suggested.** The standard scientific stack is sufficient for anything you will be asked to do.
- R is accepted if it is your stronger language.
- Script, notebook, or small project folder — whichever you prefer.

6. How submissions are evaluated

The criteria are the same across all tasks, in roughly this order of weight:

Criterion	What we look for
Correctness	The result is actually right, and you have some basis for believing so beyond the code having run without an error.
Reproducibility	Your code runs start to finish on a machine that is not yours, without manual fixing of intermediate files. Paths are not hard-coded to your desktop. We will try to run it.
Clarity	Readable code, sensible names, brief comments where a decision is non-obvious. Structure over cleverness.

Quality of reasoning	Your report shows you understood what you did. *Why* you made a choice counts for more than the choice itself.
Optional components	Sound implementation and honest interpretation, if your task includes them and you attempt them.

Two things that sink otherwise good submissions:

code that only runs on the author's own laptop, and results reported with no check that what they were computed from was sound.

One thing that reliably impresses:

noticing a problem nobody pointed you at, and saying so — including when it is a problem with your own approach.

7. Use of AI assistance

AI coding assistants are permitted. They are part of how this lab works, and pretending otherwise would be dishonest on both sides. Two conditions:

- **Disclose it** — one line in your report naming which tools you used and roughly for what.
- **Own it** — you must be able to explain every line you submit, and any design decision in it, in conversation. We may ask.

Undisclosed use, or code you cannot explain, is treated as an integrity problem rather than a technical shortcoming.

8. If you would like to prepare

No preparation is required, and there is nothing to revise for. If you would simply like to feel comfortable, the general areas worth being at ease with are:

- reading tabular data files and inspecting their size, types, and missing values before doing anything else
- combining tables that share an identifier column, and understanding what happens to records that fail to match
- recognising and reasoning about duplicates, missing values, and inconsistent identifiers
- basic descriptive statistics, and one standard error metric if your task involves prediction
- writing a short document that a stranger could follow

If several of these are unfamiliar, that is fine — say so when you submit. We would rather know your real starting point than receive a polished submission that does not reflect it.

9. Questions

Direct any questions to Prof. Chen. Clarifying questions are welcome at any point before your deadline.

Last updated: 20 August 2026